

Paradigma de Linguagem de Programação – PLP

Exercício Aula 9

Para cada trecho: (a) qual é a saída? (b) a decisão foi tomada na compilação ou na execução?

1 · Java

The screenshot shows the OneCompiler Java IDE. The code in Main.java is as follows:

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         class Animal {
6
7             String som() { return "..."; }
8
9         }
10
11         class Cachorro extends Animal {
12
13             String som() { return "au"; }
14
15         }
16
17         Animal x = new Cachorro();
18
19         System.out.println(x.som());
20     }
21 }
```

The console output on the right shows "au".

a) au

b) É execução, já que, embora x seja declarado como Animal, o objeto real é um Cachorro. Métodos de instância em Java usam dispatch dinâmico.

2 · C++

The screenshot shows the OneCompiler C++ IDE. The code in Main.cpp is as follows:

```
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     struct A {
7
8         void f() { cout << "A"; }
9
10    };
11
12    struct B : A {
13
14        void f() { cout << "B"; }
15
16    };
17
18    B b; A *p = &b;
19
20    p->f();
21 }
```

The console output on the right shows "A".

a) A

b) É compilação. Pois f() não é virtual em A. Portanto, o C++ resolve a chamada usando o tipo estático do ponteiro (A*). Se fosse (virtual void f()), a saída seria B, com despacho dinâmico.

3 · Java



The screenshot shows the OneCompiler Java IDE. The code in Main.java is as follows:

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         class A { String nome = "A";
6
7         String getNome() { return nome; } }
8
9         class B extends A { String nome = "B";
10
11         String getNome() { return nome; } }
12
13         A x = new B();
14
15         System.out.println(x.nome + " " + x.getNome());
16     }
17 }
```

The console output on the right is "A B".

a) A B

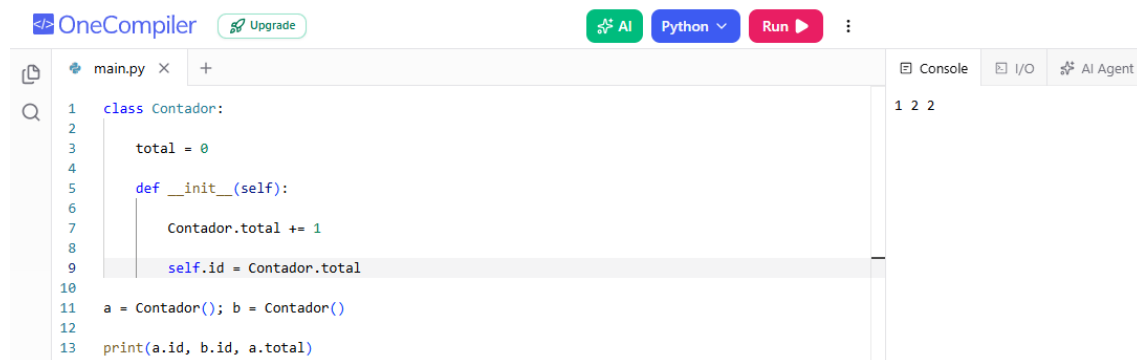
b) Aqui aparecem dois comportamentos diferentes:

x.nome → campos não são polimórficos em Java. Como x é A, usa A.nome → A.

x.getNome() → métodos de instância são polimórficos. O objeto é B, então chama B.getNome() → B.

Portanto, campo é compilação, já método é execução.

4 · Python



The screenshot shows the OneCompiler Python IDE. The code in main.py is as follows:

```
1 class Contador:
2
3     total = 0
4
5     def __init__(self):
6
7         Contador.total += 1
8
9         self.id = Contador.total
10
11 a = Contador(); b = Contador()
12
13 print(a.id, b.id, a.total)
```

The console output on the right is "1 2 2".

a) 1 2 2

b) o contador é incrementado duas vezes:

a.id = 1

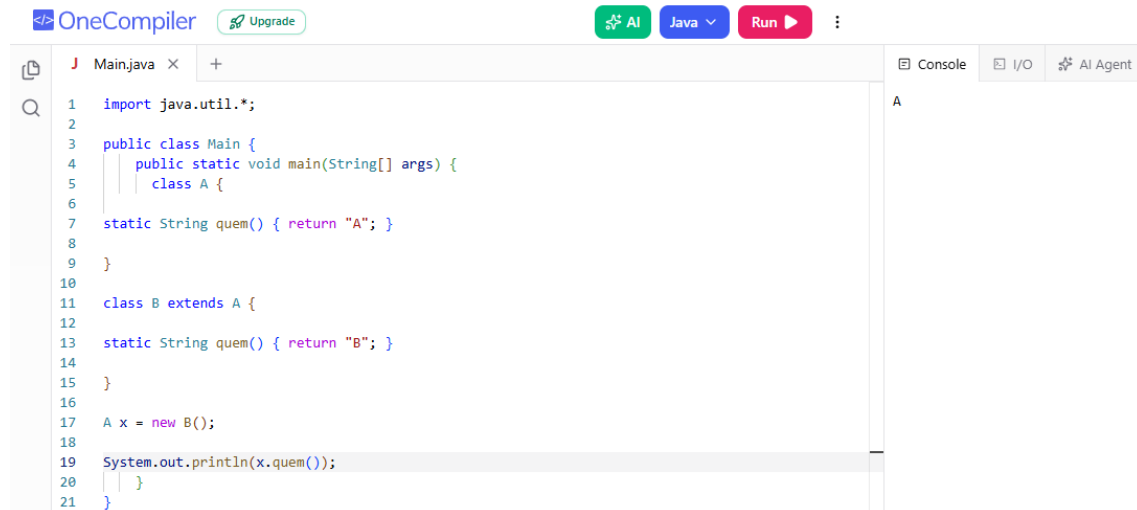
b.id = 2

Contador.total = 2

a.total procura o atributo no objeto e, não encontrando, encontra o atributo da classe.

Portanto é Execução, já que, Python faz a resolução de atributos/métodos dinamicamente.

5 · Java



The screenshot shows the OneCompiler Java IDE with a file named 'Main.java'. The code defines a base class 'A' with a static method 'quem()' that returns 'A'. A subclass 'B' extends 'A' and overrides 'quem()' to return 'B'. In the 'main' method, an instance of 'B' is created and the 'quem()' method is called on it. The output in the console is 'A'.

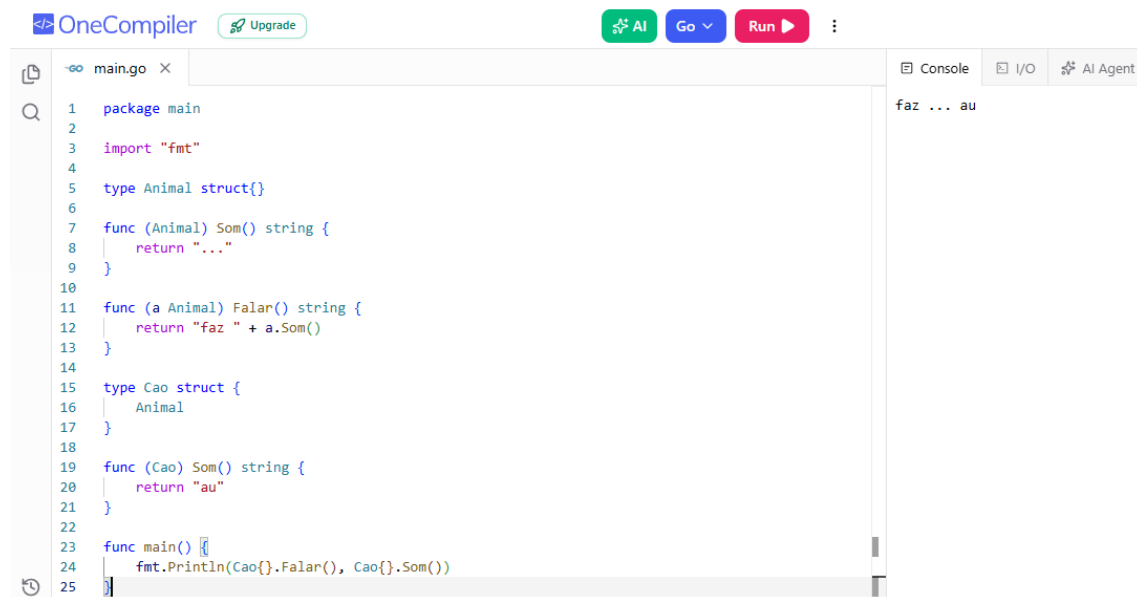
```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         class A {
6
7             static String quem() { return "A"; }
8
9         }
10
11         class B extends A {
12             static String quem() { return "B"; }
13
14         }
15
16         A x = new B();
17
18         System.out.println(x.quem());
19     }
20 }
21 }
```

Console: A

a) A

b) quem() é static. Métodos estáticos não são sobrescritos de forma polimórfica, mas sim eles são ocultados (method hiding). A chamada é determinada pelo tipo da referência e, portanto, Decisão na compilação static

6 · Go



The screenshot shows the OneCompiler Go IDE with a file named 'main.go'. The code defines a base struct 'Animal' with a method 'Som()' that returns '...'. A struct 'Cao' embeds 'Animal' and overrides 'Som()' to return 'au'. In the 'main' function, the 'Falar()' method of 'Cao' is called, which internally calls 'Som()'. The output in the console is 'faz ... au'.

```
1 package main
2
3 import "fmt"
4
5 type Animal struct{}
6
7 func (Animal) Som() string {
8     return "..."
9 }
10
11 func (a Animal) Falar() string {
12     return "faz " + a.Som()
13 }
14
15 type Cao struct {
16     Animal
17 }
18
19 func (Cao) Som() string {
20     return "au"
21 }
22
23 func main() {
24     fmt.Println(Cao{}.Falar(), Cao{}.Som())
25 }
```

Console: faz ... au

a) faz ... au

b) Em Go, isso não funciona como o polimorfismo dinâmico de Java. A incorporação Estático ou (embedding) promove métodos, mas não transforma automaticamente as chamadas internas em despacho virtual.